

CS F407: Artificial Intelligence Lab — Week 5 Report

Transformers, Self-Attention Mechanics, and Autoregressive Language Modeling

Author: Samar Talwar

Course: CS F407 (Artificial Intelligence), BITS Pilani Goa Campus

Date: Semester 1, AY 2026-27

1. Executive Summary & Mathematical Framework

This report documents the architectural design, mathematical derivations, empirical validation, and training results for **TinyGPT**, an autoregressive decoder-only Transformer Language Model built from scratch in PyTorch and NumPy without high-level library abstractions.

1.1 Autoregressive Factorisation

Language modeling factorises the joint probability of a sequence of discrete tokens $(x_1, x_2, \dots, x_T)$ from a finite vocabulary $\mathcal{V}$ via the exact chain rule of probability: $P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, x_2, \dots, x_{t-1})$

Unlike fixed-order N -gram models that truncate context $P(x_t | x_{<t}) \approx P(x_t | x_{t-N+1:t-1})$, decoder-only Transformers maintain an adaptive full-context representation over a context window of length T using scaled dot-product attention and Pre-LayerNorm residual blocks.

2. Theoretical Foundations & Architectural Derivations

2.1 Scaled Dot-Product Attention

Given an input sequence $X \in \mathbb{R}^{T \times d_{\text{model}}}$, self-attention computes query, key, and value representations via learned linear projections: $Q = XW_Q$, $K = XW_K$, $V = XW_V$ where $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

The attention weights and context-aggregated values are computed as: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$

Mathematical Derivation of Variance Scaling ($\sqrt{d_k}$)

Let $q, k \in \mathbb{R}^{d_k}$ be independent query and key vectors whose components are independent random variables with zero mean and unit variance: $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$, $\text{Var}(q_i) = \text{Var}(k_i) = 1$.

The dot product is $S = q^T k = \sum_{i=1}^{d_k} q_i k_i$. 1. **Expected Value:** $\mathbb{E}[S] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$ 2. **Variance:** $\text{Var}(S) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} (\mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2)$ Since q_i, k_i are independent: $\mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = (1)(1) = 1 \implies \text{Var}(S) = \sum_{i=1}^{d_k} 1 = d_k$ Therefore, standard deviation scales as $\sigma_S = \sqrt{d_k}$.

3. **Scaled Dot Product:** $\text{Var}\left(\frac{S}{\sqrt{d_k}}\right) = \frac{1}{d_k} \text{Var}(S) = \frac{d_k}{d_k} = 1.0$

Why this matters: Without the $1/\sqrt{d_k}$ scaling factor, for large head dimensions (e.g. $d_k = 64$ or 512), the magnitude of QK^T grows large. Large logits feed into the softmax function push it into regions of near-zero gradients (saturation), severely impeding backpropagation.

2.2 Causal Attention Masking

To preserve the autoregressive property during parallel training, token t must only attend to tokens $\leq t$. Future information is suppressed via an additive causal mask $M \in \mathbb{R}^{T \times T}$: $M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$

When computing softmax: $\text{softmax}(A + M)_{ij} = \frac{\exp(A_{ij} + M_{ij})}{\sum_{k=1}^T \exp(A_{ik} + M_{ik})}$ For $j > i$, $A_{ij} + M_{ij} = -\infty$, yielding $\exp(-\infty) = 0$. Consequently, future tokens receive strictly zero attention weight.

2.3 Multi-Head Attention (MHA)

Rather than performing a single attention function in d_{model} space, Multi-Head Attention projects queries, keys, and values into H distinct subspaces of dimension d_{head} : $\text{head}_h = \text{Attention}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)})$, $h \in \{1, \dots, H\}$
 $\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$ where $W_O \in \mathbb{R}^{(H \cdot d_{\text{head}}) \times d_{\text{model}}}$.

Scaling Factor vs Head Dimension Rule

- **Scaling Factor:** $1/\sqrt{d_k}$ depends strictly on the projection dimension d_{head} of individual queries and keys, not on d_{model} or H .
 - **Standard Convention:** Typically $d_{\text{head}} = d_{\text{model}}/H$ so that the concatenated dimension $H \cdot d_{\text{head}} = d_{\text{model}}$ and $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$.
 - **Non-Standard Feasibility:** $H \cdot d_{\text{head}}$ is not required to equal d_{model} . For example, with $d_{\text{model}} = 8$, $H = 6$, and $d_{\text{head}} = 2$, the concatenated output has dimension $6 \times 2 = 12$. The projection matrix $W_O \in \mathbb{R}^{12 \times 8}$ linearly maps the 12-dimensional concatenated features back to $d_{\text{model}} = 8$ without mathematical contradiction.
-

2.4 Pre-LayerNorm Transformer Block Architecture

Modern Transformer decoder blocks apply Pre-LayerNorm (LayerNorm before attention and FFN) rather than Post-LayerNorm:

1. **Self-Attention Sub-Layer:** $x^{(1)} = x + \text{MHA}(\text{LayerNorm}(x))$ 2. **Feed-Forward Sub-Layer:** $x^{(2)} = x^{(1)} + \text{FFN}(\text{LayerNorm}(x^{(1)}))$ where:

$$\text{FFN}(z) = W_2 \text{ReLU}(W_1 z + b_1) + b_2, \quad W_1 \in \mathbb{R}^{d_{\text{model}} \times 4d_{\text{model}}}, \quad W_2 \in \mathbb{R}^{4d_{\text{model}} \times d_{\text{model}}}$$

Pre-LayerNorm provides an unobstructed residual stream ($x + f(x)$), stabilizing gradient flow across deep networks and eliminating the need for warm-up learning rate schedules.

2.5 Analytical Parameter Count Derivation

For TinyGPT configured with: - Vocabulary Size $V = 25$ - Embedding Dimension $d_{\text{model}} = 48$ - Block Size $T = 32$ - Number of Heads $H = 4$ ($d_{\text{head}} = 48/4 = 12$) - Transformer Layers $L = 2$

Exact Layer Breakdown:

1. **Token Embeddings:** $N_{\text{tok}} = V \times d_{\text{model}} = 25 \times 48 = 1,200$
2. **Position Embeddings:** $N_{\text{pos}} = T \times d_{\text{model}} = 32 \times 48 = 1,536$
3. **Per Transformer Block:** - MHA Projections (Q, K, V across 4 heads): $3 \times H \times (d_{\text{model}} \times d_{\text{head}}) = 3 \times 4 \times (48 \times 12) = 6,912$ - MHA Output Projection (W_O + bias): $(H \cdot d_{\text{head}}) \times d_{\text{model}} + d_{\text{model}} = (48 \times 48) + 48 = 2,352$ - LayerNorm 1 (γ, β): $2 \times d_{\text{model}} = 2 \times 48 = 96$ - FFN Layer 1 ($W_1 + b_1$): $d_{\text{model}} \times (4d_{\text{model}}) + 4d_{\text{model}} = (48 \times 192) + 192 = 9,216 + 192 = 9,408$ - FFN Layer 2 ($W_2 + b_2$): $(4d_{\text{model}}) \times d_{\text{model}} + d_{\text{model}} = (192 \times 48) + 48 = 9,216 + 48 = 9,264$ - LayerNorm 2 (γ, β): $2 \times d_{\text{model}} = 2 \times 48 = 96$ - **Total per Block:** $N_{\text{block}} = 6,912 + 2,352 + 96 + 9,408 + 9,264 + 96 = 28,128$
4. **Final LayerNorm (γ, β):** $N_{\text{ln_f}} = 2 \times d_{\text{model}} = 96$
5. **LM Head ($W_{\text{head}} + b_{\text{head}}$):** $N_{\text{lm}} = d_{\text{model}} \times V + V = 48 \times 25 + 25 = 1,200 + 25 = 1,225$

Total Analytical Parameters:

$$N_{\text{total}} = N_{\text{tok}} + N_{\text{pos}} + L \times N_{\text{block}} + N_{\text{ln_f}} + N_{\text{lm}} \quad N_{\text{total}} = 1,200 + 1,536 + (2 \times 28,128) + 96 + 1,225 = 60,313$$

The empirical parameter count from `TinyGPT.count_parameters()` matches this derivation exactly (60,313).

3. Experimental Analysis & Numerical Verification

All quantitative values presented in this section are extracted directly from the unrounded machine outputs in `week05_transformers_ar/results/`.

3.1 Attention Variance Scaling Verification

From `results/attention_analysis.json`: We sampled $N = 3,000$ independent pairs $q, k \sim \mathcal{N}(0, I_d)$ across dimensions $d \in [2, 8, 32, 128, 512]$ and evaluated empirical standard deviation $\sigma(q^\top k)$:

Dimension (d)	Empirical Std (σ)	Theoretical $\sqrt{d}$	Ratio $\sigma/\sqrt{d}$
2	1.408254	1.414214	0.995786
8	2.845659	2.828427	1.006093
32	5.529509	5.656854	0.977488
128	11.418496	11.313708	1.009262
512	22.655445	22.627417	1.001239

Across all dimensions, the empirical ratio $\sigma/\sqrt{d}$ remains strictly within $[0.977, 1.009]$, confirming the theoretical derivation.

Unscaled vs. Scaled Attention Variance

Testing scaled scores $S/\sqrt{d_k}$ versus unscaled scores S for head dimensions $d_k \in [4, 8, 16, 32, 64]$:

Head Dim (d_k)	Scaled Std $\sigma(S/\sqrt{d_k})$	Unscaled Std $\sigma(S)$	Variance Ratio
4	0.918966	1.837933	2.0000
8	1.109610	3.138451	2.8284 ($\sqrt{8}$)
16	0.986494	3.945976	4.0000 ($\sqrt{16}$)
32	0.978238	5.533747	5.6569 ($\sqrt{32}$)
64	1.009576	8.076611	8.0000 ($\sqrt{64}$)

Scaling preserves score standard deviation near **1.0** regardless of head dimension.

3.2 Training Dynamics & Convergence

From `results/training_loss.json`: TinyGPT was trained on the repeated canonical corpus for 180 iterations using AdamW (`lr = 3 × 10-3`, `batch_size = 32`, `block_size = 32`):

Step	Evaluation Loss	Notes / Training Stage
0	3.384801	Random initialization ($\approx \ln 25 = 3.2189$)
20	2.061595	Rapid character frequency and bigram acquisition
40	1.617704	Matching baseline bigram loss (1.6013)
60	1.284008	Acquiring multi-character token sequences
80	0.900996	Resolving phrase-level syntax
100	0.587539	Learning sentence beginnings and transitions
120	0.345517	Long-range context stabilization
140	0.257031	Fine-grained punctuation and ending structure
160	0.178116	High-confidence sentence reproduction
180	0.159833	Fully converged ($> 95.2\%$ loss reduction)

- **Initial Loss:** 3.3848
- **Final Loss:** 0.1598
- **First-10 Step Mean Loss:** 2.8162

- **Last-10 Step Mean Loss:** 0.1586

3.3 Temperature Scaling in Autoregressive Generation

From `results/generation_samples.json`: We evaluated autoregressive generation across temperatures $\tau \in \{0.3, 0.7, 0.8, 1.0, 2.0\}$ using prompt "baana ", "attention ", and an empty prompt "":

Temperature (τ)	Sample Continuation (Prompt: " ")	Qualitative Characteristics
0.3	\ngeneration repeats next token prediction.\n\n\nto learn is to c	Highly deterministic, exact grammatical corpus sentence reconstruction.
0.7	\ngenextion lets a token a loook at arlier teans.\na trans to n	Coherent, minor stochastic character substitutions.
0.8	\ngenextion lets a token a loook at arlier teans.\na trans thex	Moderate creativity with slight syntactic degradation.
1.0	\ng model learns from edict tokenns rer prediction.\n\ngeneration	High variance, frequent non-words and spelling splices.
2.0	\ng moswe ptrwe mpreduner p by pexxts ticktieng\ntioonfokernfri	Near-uniform entropy; complete breakdown into character noise.

3.4 Bigram Baseline vs. TinyGPT Comparison

From `results/comparison.json`:

Feature / Metric	Bigram Model	TinyGPT
Architecture	First-order counting table $P(x_t x_{t-1})$	2-Layer Decoder Transformer
Context Length (T)	1 character	32 characters
Trainable Parameters	625 (25×25)	60,313
Cross-Entropy Loss	1.6013	0.1598
Syntactic Coherence	False (degenerate repeating loops, e.g. by token token token...)	True (coherent grammatical sentences)
Verbatim Sentence Recall	0.00%	High (> 16.7% exact matching on short sampled prompts)

4. Addressing Common Misconceptions

- The Fixed-Slicing Fallacy:** *Misconception:* Multi-head attention simply slices the input tensor into H chunks along the feature dimension.
Reality: Heads are **learned linear projections** from the full d_{model} -dimensional space into distinct subspaces ($Q = XW_Q^{(h)}$). Slicing without projection would prevent heads from learning different linear combinations of input features.
- Attention Weights vs. Value Vectors:** *Misconception:* Attention weights represent the output features of the attention block.
Reality: Attention weights $\alpha_{ij} \in [0, 1]$ form a normalized scalar weighting matrix indicating *how much* token i attends to token j . The output is a linear combination of *Value vectors* ($z_i = \sum_j \alpha_{ij} v_j$).
- Query/Key/Value Sources:** *Misconception:* Queries, keys, and values come from three different external sources.
Reality: In decoder self-attention, Q, K, V are all linear projections of the **same input tensor X** . In encoder-decoder cross-attention, Q comes from the decoder while K, V originate from the encoder.
- Causal Masking during Training vs. Inference:** *Misconception:* Causal masking is used during inference to generate one token at a time.

Reality: Causal masking is used **during parallel training** to allow all T next-token predictions to be computed in a single forward pass while preventing future leakage. During autoregressive generation, generation is inherently sequential: token t is computed from previous tokens $1 \dots t - 1$.

5. Student Reflection & "Think About It" Analysis

Reflection 1: Self-Attention vs. N-Gram Contextual Capacity

Question: Why does self-attention enable capturing long-range dependencies that N-gram models fundamentally fail on?

Answer:

Self-attention enables capturing arbitrary long-range dependencies by computing content-dependent pairwise affinity scores $\alpha_{ij} \propto q_i^\top k_j$ across the entire context window in $O(1)$ path length, allowing direct information routing between distant tokens without passing through intermediate states. In contrast, fixed N -gram language models suffer from combinatorial parameter explosion requiring $|\mathcal{V}|^N$ entries, which becomes physically intractable for long contexts (for $|\mathcal{V}| = 25$ and $N = 32$, requiring $\approx 5.4 \times 10^{44}$ parameters). While the baseline bigram model with 625 parameters plateaued at a training cross-entropy loss of **1.6013** (results/comparison.json, bigram_model.training_loss), the 60,313-parameter TinyGPT architecture with context length $T = 32$ successfully converged to a final loss of **0.1598** (results/comparison.json, tiny_gpt.final_loss). Furthermore, embedding continuous representations allows transformers to generalize across novel semantic contexts, avoiding the catastrophic zero-probability sparsity that invalidates unseen N -gram prefixes.

Reflection 2: Causal Masking Mechanics & Training Parallelism

Question: What is the mechanical role of the causal mask during training versus during autoregressive inference?

Answer:

During training, causal masking enables fully parallelized sequence processing by applying an upper-triangular additive mask $M_{ij} = -\infty$ for $j > i$, ensuring that $\exp(-\infty) = 0$ in the softmax computation and strictly preventing position i from attending to future tokens $j > i$. This upper-triangular masking structure (results/attention_analysis.json, numpy_causal_attention_demo.causal_upper_triangle_is_zero: true) permits all T next-token prediction losses to be computed simultaneously across the entire sequence in a single forward pass without information leakage. During autoregressive inference, token generation is inherently sequential: the model evaluates the current prefix $x_{1:t}$, samples the next token x_{t+1} from the final logit position, and appends it to the context. The causal mask guarantees that the contextual representation of token x_i computed during parallel training remains strictly identical to the representation generated when evaluating token x_i during sequential inference.

Reflection 3: Temperature Scaling Dynamics & Entropy

Question: Why does temperature $\tau < 1.0$ sharpen probability distributions, and what are the failure modes of extreme temperature settings ($\tau \rightarrow 0$ vs $\tau \rightarrow \infty$)?

Answer:

Temperature scaling modulates the sharpness of the categorical probability distribution by scaling pre-softmax logits by a positive factor τ , yielding probabilities $P(i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$. When $\tau < 1.0$, logit differentials are magnified ($(z_i - z_j)/\tau > z_i - z_j$), concentrating probability mass onto the maximum logit, lowering entropy, and producing deterministic, low-perplexity continuations (results/generation_samples.json, temperature: 0.3). In the extreme limit $\tau \rightarrow 0$, the distribution collapses into greedy argmax selection, which ensures local syntactic correctness but often induces repetitive loops. Conversely, as $\tau \rightarrow \infty$, scaled logits approach zero ($z_i/\tau \rightarrow 0$), causing $\exp(z_i/\tau) \rightarrow 1.0$ and degenerating into a maximum-entropy uniform distribution $P(i) = 1/|\mathcal{V}|$ that emits unstructured random noise (results/generation_samples.json, temperature: 2.0).

6. Verification Suite & Mutation Testing Summary

The test suite in week05_transformers_ar/tests/test_transformers.py contains 15 automated test cases: 1. **NumPy vs.**

PyTorch Oracle: Verified that causal_self_attention_numpy and PyTorch Head produce identical attention weights and output tensors on shared fixed weights ($< 10^{-6}$ error). 2. **Analytical Parameter Oracle:** Verified that TinyGPT.count_parameters()

matches the 60,313 analytical derivation. 3. **Causal Invariance Test:** Mutating future tokens in input sequences verified zero change in prefix logits ($< 10^{-6}$ absolute deviation). 4. **Attention Score Variance Scaling Test:** Verified that $\sigma(q^T k / \sqrt{d})$ stays near **1.0** across head dimensions. 5. **Temperature & Entropy Invariant Tests:** Verified strict monotonic entropy growth with increasing τ and deterministic argmax at $\tau \leq 10^{-6}$. 6. **Training Convergence Test:** Verified that 180 steps of AdamW drops cross-entropy loss from > 3.0 to < 0.50 (empirical result: **0.1598**). 7. **Real Module Monkeypatch Mutation Tests:** - *Mutation 1 (Corrupted Unmasked Attention):* Monkeypatching `Head.forward` to remove the causal mask fails the future-leakage test ($> 10^{-4}$ deviation). - *Mutation 2 (Missing $\sqrt{d_k}$ Scaling):* Monkeypatching `Head.forward` to remove scaling fails the variance scaling assertion ($\sigma_{\text{unscaled}} > 2.0 \times \sigma_{\text{scaled}}$). - *Mutation 3 (Broken Residual Connection):* Monkeypatching `TransformerBlock.forward` to omit $x + \text{sa}(x)$ fails the residual pass-through identity test.